

CS 240 - Lab 2

TAs: Yash J. & Abhi Jain

Spring 2025

Welcome to the second graded lab of this course!!

As discussed in the last lab, the in-lab portion would be based on/similar to the outlab of the previous lab. Whereas, the outlab portion will build on different techniques iterative optimization techniques like gradient descent, stochastic gradient descent, Newton's second order method, etc.

They are arranged in increasing order of difficulty, and you are encouraged to attempt them sequentially. Throughout the lab, focus on understanding what is happening and why, and feel free to pause and ask questions rather than mechanically completing the tasks and moving on.

Instructions:

- This lab has 3 questions. Only the first question is graded in-lab. We have provided you with the following files:

```
lab2.pdf
q1/
|-- q1_solve.py
|-- q1_main.py
|-- q1_train.csv
|-- q1_test.csv
```

- Create a folder on your Desktop with the following directory structure. The folder must contain only the one specified file:

```
submission_<roll_number>/
|-- q1_solve.py
```

- Once you have completed the work, open the terminal, type the following command, and then call a TA: `check-cs240`
- The TA will run `submit-cs240` to submit your work to the server. **It is your responsibility to ensure your work is submitted.** If you fail to inform us to submit your work, you will not be graded.

Graded In-Lab Begins :)

Q1. In-Lab: Ridge Regression and Hyperparameter Optimization

*You are a data scientist at SunGrid, a renewable energy startup that claims to be “AI-powered” because NumPy is installed on the server. Your task is to predict **daily energy demand** using environmental signals such as temperature and humidity, and a few extra features added by Dr. Jhatka during an enthusiastic brainstorming session. No one remembers what these features mean, but deleting them is considered “too risky for production”.*

*Last month, Motu trained an ordinary least squares regression model. The training error was almost zero. The test error was so bad that the battery system briefly attempted to violate the laws of physics. Patlu suggested using **Ridge Regression** to fix the model*

You are now tasked with fixing the model as per Patlu’s suggestions and tuning the parameters using cross-validation.

The dataset is small and the features are highly correlated. Using Ordinary Least Squares (OLS), the model fits the training data well but performs poorly on unseen data.

Your task is to stabilize the model using **Ridge Regression**, select the regularization parameter λ using **cross-validation** and compare simple optimization strategies for hyperparameter tuning.

Data Format

You are given two CSV files:

- q1_train.csv
- q1_test.csv

Each row has the form: $(x_1, x_2, \dots, x_d, y)$ where the last column is the target value y .

Linear Regression Review

Given a design matrix $X \in \mathbb{R}^{N \times d}$ and targets $y \in \mathbb{R}^N$, Ordinary Least Squares solves:

$$\min_w \|y - Xw\|_2^2$$

If $X^T X$ is invertible, the solution is:

$$w_{\text{OLS}} = (X^T X)^{-1} X^T y$$

OLS is unstable when features are correlated, or $X^T X$ is ill-conditioned. You have studied Ridge Regression now, and hence the graded tasks for your lab are:

Task 1 Implementing Ridge Regression (20 Points)

Ridge regression adds an ℓ_2 penalty:

$$\min_w \|y - Xw\|_2^2 + \lambda \|w\|_2^2$$

The closed-form solution is:

$$w = (X^T X + \lambda I)^{-1} X^T y$$

Task 1.1 Feature Standardization

Before applying ridge regression, each feature is standardized:

$$x_j \leftarrow \frac{x_j - \mu_j}{\sigma_j}$$

where μ_j and σ_j are computed on the training set.

Task 1.2 Handling Bias Term

When a bias (intercept) term is included by adding a column of ones to X , it should *not* be regularized. This is handled by using:

$$D = \text{diag}(0, 1, 1, \dots, 1)$$

and solving:

$$(X^T X + \lambda D)w = X^T y$$

Task 2 Implementing Cross-Validation (15 Points)

To estimate generalization performance, you will use k -fold cross-validation; i.e. split the data into k folds, train on $k - 1$ folds, validate on the remaining fold and average this MSE over all folds.

Task 3 Optimizing Hyperparameters (15 Points)

Task 3.1 Grid Search

Grid search evaluates a fixed set of λ values (log-spaced): $\lambda \in \{10^{-4}, \dots, 10^4\}$

It is simple and exhaustive but can be computationally inefficient.

Task 3.2 Random Search

Instead of fixed points, random search samples: $\lambda = 10^u$, $u \sim \text{Uniform}(a, b)$

Random search often finds good hyperparameters faster than grid search, especially when the search space is large.

What You Must Implement?

In `q1_solve.py`, implement all the non-implemented functions.

DO NOT:

- Don't use scikit-learn in `q1_solve.py`.
- Don't make any changes in `q1_main.py`. We will change it our autograding and only `q1_solve.py` has to be submitted.

Graded In-Lab Ends :)

Exercises on Optimization Begin :O

Note: This contains the ungraded in-lab and the graded out-lab.

Preliminaries and Setup

This section establishes the formal notation and mathematical properties required to solve the assignment. You are expected to interpret these definitions to implement the algorithms and analyze their convergence behaviors.

1. Least Squares Linear Regression

We consider the **Least Squares** problem for Linear Regression.

- **Dataset:** Let $X \in \mathbb{R}^{n \times d}$ be the feature matrix containing n samples and d features. Let $y \in \mathbb{R}^n$ be the target vector.
- **Parameters:** Let $w \in \mathbb{R}^d$ be the weight vector we wish to optimize.
- **Objective Function:** We minimize the Mean Squared Error (MSE): $f(w) = \frac{1}{n} \|Xw - y\|_2^2$

2. Linear Algebra Foundations

Understanding the geometry of X is crucial for analyzing optimization performance.

2.1 Singular Value Decomposition (SVD)

Any matrix $X \in \mathbb{R}^{n \times d}$ can be decomposed as $X = U\Sigma V^\top$, where:

- $U \in \mathbb{R}^{n \times n}$ and $V \in \mathbb{R}^{d \times d}$ are orthonormal matrices, i.e. $U^\top U = I$ and $V^\top V = I$.
- The matrix $\Sigma \in \mathbb{R}^{n \times d}$ is diagonal (possibly rectangular), with diagonal entries

$$\Sigma_{ii} = \sigma_i, \quad \sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(n,d)} \geq 0,$$

and $\Sigma_{ij} = 0$ for $i \neq j$. The values $\{\sigma_i\}$ are called the **singular values** of X .

- **Geometric interpretation:** X maps the unit sphere to an ellipsoid whose principal axes are given by the right singular vectors (columns of V) and whose axis lengths are $\{\sigma_i\}$.
- **Numpy:** `U, Σ, VT = np.linalg.svd(X, full_matrices=False)`

Computational Cost: Computing the SVD or inversion for the pseudoinverse scales as $\mathcal{O}(nd^2)$ or $\mathcal{O}(d^3)$. For large d (e.g., $d > 10,000$), this is **intractable**, motivating the use of iterative methods.

2.2 Eigenvalues and The Hessian

For Least Squares, the curvature of the loss surface is determined by the hessian matrix $H = X^\top X$.

- The eigenvalues of H satisfy $\lambda_i(X^\top X) = \sigma_i(X)^2$. This is because the SVD diagonalizes the Gram matrices:

$$X^\top X = V\Sigma^\top \Sigma V^\top, \quad XX^\top = U\Sigma \Sigma^\top U^\top.$$

Hence, the squared singular values $\{\sigma_i^2\}$ are the eigenvalues of both $X^\top X$ and $XX^\top$.

- Curvature of the loss surface along direction v_i (eigenvector of $X^\top X$) is proportional to σ_i^2 .
- **Numpy:** `eigenvalues = np.linalg.eigvals(X)` (Use `eigvalsh` for symmetric matrices)

2.3 Condition Number (κ)

The condition number measures how “stretched” the loss landscape is (elliptical vs. spherical).

$$\kappa(X) = \frac{\sigma_{\max}(X)}{\sigma_{\min}(X)} \implies \kappa(X^\top X) = \frac{\lambda_{\max}(X^\top X)}{\lambda_{\min}(X^\top X)} = \kappa^2(X)$$

- **Numpy:** `np.linalg.cond(X)`

2.4 Stable Rank (sr)

The stable rank provides a continuous notion of effective dimensionality:

$$\text{sr}(X) = \frac{\sum \sigma_i^2}{\sigma_{\max}^2}$$

3. Relevant Numpy Functions Cheat Sheet

Mathematical Concept	Numpy Implementation	Note
Transpose	<code>A.T</code>	View, no copy
Matrix Multiplication AB	<code>A @ B</code>	
Solve $Ax = b$ (Exact)	<code>np.linalg.solve(A, b)</code>	Requires A square/full rank
Solve Least Squares	<code>np.linalg.lstsq(X, y, rcond=None)</code>	The “intractable” baseline
Pseudo-inverse	<code>np.linalg.pinv(X)</code>	SVD-based
Singular values	<code>np.linalg.svd(X)</code>	
Eigenvalues (general)	<code>np.linalg.eigvals(A)</code>	May be complex
Eigenvalues (symmetric)	<code>np.linalg.eigvalsh(A)</code>	Faster, real-valued
Condition number	<code>np.linalg.cond(X)</code>	Ill-conditioning measure
Matrix rank	<code>np.linalg.matrix_rank(X)</code>	Numerical rank
Gradient $\nabla f(w)$ for OLS	<code>X.T @ (X @ w - y)</code>	Vectorized implementation

Q2. Gradient Descent: The First Example [Ungraded In-Lab]

We will implement gradient descent on the following miniature functions:

1. Rosenbrock Function

$$f(x, y) = (a - x)^2 + b(y - x^2)^2$$

take $a = 1$ and $b = 100$

2. Rotated Anisotropic Quadratic

$$f(x) = x^T Q x - b^T x$$

You would do the following:

Task 1 Rosenbrock function

For Rosenbrock function, fill the corresponding rosenbrock callable class in func.py in functions folder. Note the MSE obtained and the minima point x^* at the end of the run. Compare it with the analytic solution obtained by putting $\nabla_x f(x) = 0$

Task 2 Rotated Anisotropic Quadratic

You repeat the same thing as above for $Q = U \Sigma V^T$, where all U, V , and Σ are 5x5 matrices.

$$U = \begin{bmatrix} -0.5615 & -0.0142 & 0.3363 & -0.7292 & 0.1992 \\ 0.6419 & -0.1191 & -0.2419 & -0.4509 & 0.5585 \\ 0.4427 & 0.3283 & 0.8318 & 0.0191 & -0.0630 \\ 0.0350 & -0.9228 & 0.3452 & 0.1621 & 0.0432 \\ 0.2748 & -0.1621 & -0.1317 & -0.4882 & -0.8016 \end{bmatrix},$$

$$V = \begin{bmatrix} 0.3582 & -0.7321 & 0.2857 & 0.4717 & 0.1777 \\ 0.2772 & 0.3230 & 0.0332 & -0.0691 & 0.9017 \\ 0.1081 & 0.4481 & 0.8425 & 0.1824 & -0.2108 \\ 0.0002 & 0.3732 & -0.3938 & 0.8382 & -0.0550 \\ 0.8850 & 0.1404 & -0.2289 & -0.1918 & -0.3286 \end{bmatrix},$$

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & 0 & 0 & 0 \\ 0 & \sigma_2 & 0 & 0 & 0 \\ 0 & 0 & \sigma_3 & 0 & 0 \\ 0 & 0 & 0 & \sigma_4 & 0 \\ 0 & 0 & 0 & 0 & \sigma_5 \end{bmatrix}.$$

The σ_i s are deliberately left empty so that you tinker with it. Try observing MSE at 10, 100, 1000 and 10000 epochs in the following cases:

1. when σ_{max} is very small or 0.
2. when for some j , $\sigma_j = 100$ and otherwise $\sigma_i = 1 \quad \forall i \neq j$.
3. when all σ_i are equal, and say equal to 10.

In all the above experiments, maintain $b = 0$.

Also experiment with the learning rate. Try to observe how MSE and convergence of the algorithm changes. Try to see if there is any correlation with condition number (κ) or stable rank (sr). Also compare x^* obtained with the analytic solution.

Running q2.py outputs only metrics_params.json for the given parameters of a , b , Q , and b^T , which function and epochs/iterations to run. It does not generate plots, you have to look into the metrics_params.json file in the q2 folder or make your own plots.

Also, note that all code is vectorised in the repository. So code accordingly.

Update!! Some people complained about convergence challenges for part B. You have write functions in func.py as if $U \neq V$. But in the code we have set $U = V$ now to ensure strong convexity. Also please do not change U and V values in the code.